Functions: Three More Features

Default values, keyword arguments, and variable-length *args / **kwargs make functions flexible.

Feature 1 · Default parameter values

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Ann"))              # uses the default
print(greet("Ann", "Namaste"))   # overrides it
```

OUTPUT

```
Hello, Ann!
Namaste, Ann!
```

- Defaults are evaluated **once**, when `def` runs.
- Parameters with defaults must come **after** those without.

PITFALL Never use a mutable default like `[]` or `{}` – it is shared between calls. Use `None` and build the list inside.

Feature 2 · Keyword arguments

```
def box(width, height, label="box"):
    return f"{label}: {width}x{height}"

print(box(2, 3))                        # positional
# keywords, any order
print(box(height=3, width=2))
# mix (positional first)
print(box(2, 3, label="crate"))
```

OUTPUT

```
box: 2x3
box: 2x3
crate: 2x3
```

- In a call, positional arguments come **before** keyword ones.
- Giving one parameter twice → `TypeError: got multiple values`.
- Keywords make calls self-documenting: `connect(host="db", timeout=5)`.

Feature 3 · *args and **kwargs

```
def stats(*nums):                  # nums is a tuple
    return min(nums), max(nums), sum(nums)

def tag(name, **attrs):            # attrs is a dict
    pairs = [f'{k}="{v}"' for k, v in attrs.items()]
    return f"<{name} {' '.join(pairs)}>"

print(stats(4, 9, 2))
print(tag("a", href="/home", id="nav"))
```

OUTPUT

```
(2, 9, 15)
<a href="/home" id="nav">
```

Syntax	Collects	Type
`*args`	extra positional arguments	tuple
`**kwargs`	extra keyword arguments	dict

The names `args` / `kwargs` are convention; the `*` and `**` are what matter.

Unpacking when calling

```
nums = [3, 1, 2]
opts = {"label": "demo", "height": 5}
# list -> positional args
print(stats(*nums))
# dict -> keyword args
print(box(7, **opts))
```

OUTPUT

```
(1, 3, 6)
demo: 7x5
```

All three together

```
def connect(host, port=5432, *extra, **opts):
    return host, port, extra, opts

print(connect("db"))
print(connect("db", 6543, "ro", ssl=True))
```

OUTPUT

```
('db', 5432, (), {})
('db', 6543, ('ro',), {'ssl': True})
```

When to reach for which

Feature	Use it for
default value	optional settings with a sensible fallback
keyword argument	clear calls; skipping optional parameters
`*args`	any number of positional values (like `print`)
`**kwargs`	pass-through options, flexible configuration

REMEMBER Signature order: `def f(a, b=1, *args, **kwargs)` – required, defaults, extras, keyword extras.

TIP Use `*args` / `**kwargs` to forward everything to another function: `return other(*args, **kwargs)` – the core of decorators.